```
/* ================================================================================
   WealthFlow Allocator v1.0 — Time-Bucketed Transaction Router
   --------------------------------------------------------------------------------
   The user requirement: "each by each payments allocate the tabs with match
   dates/months/years".

   What this module does that wealthflow-autonomous.js does NOT:
     • Reads the transaction's OWN date (from the bank statement line) rather
       than today's date, so a 29-MAY-2026 transaction lands in May 2026 even
       if you import the statement on 15-JUN-2026.
     • Cross-references existing transactions to prevent duplicates beyond the
       hash-based dedup (catches the same line being imported via email AND a
       paste-share, where the hash differs slightly because of whitespace).
     • Builds a 'month' field (YYYY-MM) on every record so the existing
       per-month UI filters work without modification.
     • Handles the special case where the same transaction is reported by
       both the bank's "transaction alert" SMS and the bank's "daily summary"
       email — collapses to a single record with provenance from both.
     • Surfaces the correct tab in the UI after allocation, so the user
       SEES the transaction land where it belongs.

   Exposes:
     • window.wfAllocate(brainResult)   — main entry; supersedes direct calls
                                          to wfApplyBrainResult.
     • window.wfFindDuplicate(brainResult) — used by the email-sync module to
                                             detect retransmits.
     • window.wfNavigateToTransaction(record, module, ym) — opens the tab and
                                             scrolls/filters to the right
                                             record.
   ================================================================================ */
(function(){
    'use strict';
    if(window.WF_ALLOCATOR_LOADED)return;
    window.WF_ALLOCATOR_LOADED='1.0';

    function _db(){return window.DB||null;}
    function _get(k){try{return (_db()&&_db().get(k))||[];}catch{return [];}}
    function _set(k,v){try{if(_db())_db().set(k,v);}catch(_){}}
    function _notify(m,t){try{if(typeof window.notify==='function')window.notify(

    // ──────────────────────────── Time bucket helpers ────────────────────────────
    function bucket(ts){
        const d=new Date(typeof ts==='number'?ts:Date.parse(ts));
        if(isNaN(d.getTime()))return null;
```

```
    const y=d.getFullYear(), m=d.getMonth()+1, day=d.getDate();
    return {
        year:y, month:m, day:day,
        ym:y+'-'+String(m).padStart(2,'0'),
        ymd:y+'-'+String(m).padStart(2,'0')+'-'+String(day).padStart(2,'0'),
        iso:d.toISOString(),
        date_ms:d.getTime()
    };
}
window.wfTimeBucket=bucket;

// ———————————————————— Duplicate detection ————————————————————————
function _amountClose(a,b){return Math.abs(Number(a||0)-Number(b||0))<0.5;}
function _sameDay(a,b){
    const da=new Date(typeof a==='number'?a:Date.parse(a));
    const db=new Date(typeof b==='number'?b:Date.parse(b));
    if(isNaN(da.getTime())||isNaN(db.getTime()))return false;
    return da.getFullYear()===db.getFullYear()&&da.getMonth()===db.getMonth()
}
function _normalizeText(s){return String(s||'').toLowerCase().replace(/[^a-z0
function _fuzzyEq(a,b){
    const na=_normalizeText(a), nb=_normalizeText(b);
    if(!na||!nb)return false;
    if(na===nb)return true;
    // 80% character overlap counts as a match
    if(na.length>10&&nb.length>10){
        const shorter=na.length<nb.length?na:nb;
        const longer =na.length<nb.length?nb:na;
        return longer.includes(shorter)||shorter.length/longer.length>0.8&&_l
    }
    return false;
}
function _levRatio(a,b){
    if(a===b)return 1;
    const m=a.length, n=b.length;
    if(!m||!n)return 0;
    const dp=Array(n+1).fill(0).map((_,i)=>i);
    for(let i=1;i<=m;i++){
        let prev=dp[0]; dp[0]=i;
        for(let j=1;j<=n;j++){
            const tmp=dp[j];
            dp[j]=a[i-1]===b[j-1]?prev:1+Math.min(prev,dp[j-1],dp[j]);
            prev=tmp;
        }
    }
    return 1-dp[n]/Math.max(m,n);
}
```

```
function findDuplicate(brain){
    if(!brain||!brain.routed)return null;
    const f=brain.routed.suggested_fields||{};
    const module=brain.routed.module;
    const amt=Number(f.amount)||0;
    const ts=f.date||f.timestamp||(brain.parsed&&brain.parsed.timestamp)||Dat
    const desc=f.desc||f.source||f.name||((brain.resolved_merchant&&brain.res
    const cardL4=f.card_last4||(brain.parsed&&brain.parsed.card_last4)||null;

    // Search the right module + adjacent ones for a fuzzy match
    const moduleArr=_get(module);
    const searchSets=[moduleArr];
    // Cross-check against expenses too (sometimes a CC tx ends up logged as
    if(module!=='expenses')searchSets.push(_get('expenses'));

    for(const arr of searchSets){
        for(const ex of arr){
            if(!ex)continue;
            if(brain.hash && ex.hash===brain.hash)return {dup:ex, exact:true}
            if(!_amountClose(ex.amount,amt))continue;
            const exDate=ex.date_ms||ex.date||0;
            if(!_sameDay(exDate,ts))continue;
            if(cardL4 && ex.card_last4 && ex.card_last4!==cardL4)continue;
            const exDesc=ex.desc||ex.source||ex.name||'';
            if(_fuzzyEq(desc,exDesc))return {dup:ex, exact:false};
        }
    }
    return null;
}
window.wfFindDuplicate=findDuplicate;

// ———————————————————— Allocation core ————————————————————————————
async function allocate(brain){
    if(!brain||!brain.ok)return {ok:false,reason:'invalid brain result'};
    if(!brain.classified&&!brain.routed)return {ok:false,reason:'not classifi

    // 1. Duplicate check (beyond hash — catches whitespace/template variants
    const dup=findDuplicate(brain);
    if(dup){
        return {ok:false,reason:'duplicate',matched:dup.dup,exact:dup.exact};
    }

    // 2. Build the time bucket from the TRANSACTION's date (not today)
    const f=(brain.routed&&brain.routed.suggested_fields)||{};
    const txTs=f.date||f.timestamp||(brain.parsed&&brain.parsed.timestamp)||D
    const tb=bucket(txTs);
```

```
if(!tb){console.warn('[allocator] could not bucket timestamp:',txTs);}

// 3. Patch the suggested_fields with a proper month field + date_ms so
//    the existing per-month UI filters work. We also normalise `date`
//    to a YYYY-MM-DD string (the format the existing app uses).
const patched=Object.assign({},f,{
    date:tb?tb.ymd:new Date(txTs).toISOString().slice(0,10),
    date_ms:tb?tb.date_ms:Number(txTs),
    month:tb?tb.ym:'',
    year:tb?tb.year:undefined,
    time_bucket:tb||undefined
});

// Inject patched fields back into a cloned brain result so the
// existing wfApplyBrainResult uses our time-aware values.
const cloned=Object.assign({},brain,{
    routed:Object.assign({},brain.routed,{suggested_fields:patched})
});

// 4. Delegate writing to wfApplyBrainResult (which already knows how
//    to handle every module, semantic allocation, the quarantine, the
//    intelligence layer, FIFO reconciliation, etc.) But it must use
//    OUR patched fields (in particular, the patched date).
if(typeof window.wfApplyBrainResult!=='function'){
    return {ok:false,reason:'wfApplyBrainResult not loaded'};
}
const res=await window.wfApplyBrainResult(cloned);

// 5. Stamp the bucket on the newly created record so we can find it
if(res&&res.ok&&res.module&&res.module!=='quarantine'&&res.module!=='cc_p
    try{
        const arr=_get(res.module);
        const newest=arr[arr.length-1];
        if(newest && !newest.month){
            newest.month=tb.ym;
            newest.year=tb.year;
            newest.date_ms=tb.date_ms;
            _set(res.module,arr);
        }
    }catch(_){}
}

// 6. Refresh the UI for the affected tab
if(res&&res.ok&&res.module){
    const renderFn={
        expenses:'renderExpenses',
        income:'renderIncome',
```

```
            subscriptions:'renderSubscriptions',
            cconetime:'renderCCOneTime',
            ccinstall:'renderCCInstall',
            loans:'renderLoans',
            loan:'renderLoans',
            goal:'renderTargets',
            targets:'renderTargets'
        }[res.module];
        if(renderFn&&typeof window[renderFn]==='function'){
            try{window[renderFn]();}catch(_){}
        }
        // Dashboard always refreshes
        try{if(typeof window.renderDash==='function')window.renderDash();}cat
    }

    return Object.assign({},res||{},{time_bucket:tb});
}
window.wfAllocate=allocate;

// ———————————————————— Navigation helper ————————————————————————
function navigateToTransaction(module,ym){
    try{
        if(typeof window.showPage==='function')window.showPage(module);
        // If the app exposes a per-month filter, set it
        if(ym){
            const monthFilterId={
                expenses:'expenseMonthFilter',
                income:'incomeMonthFilter',
                subscriptions:'subscriptionMonthFilter',
                cconetime:'cconetimeMonthFilter',
                ccinstall:'ccinstallMonthFilter'
            }[module];
            if(monthFilterId){
                const el=document.getElementById(monthFilterId);
                if(el && 'value' in el){
                    el.value=ym;
                    el.dispatchEvent(new Event('change',{bubbles:true}));
                }
            }
        }
    }catch(_){}
}
window.wfNavigateToTransaction=navigateToTransaction;

// ———————————————————— Migration: stamp old records ————————————————
// Some older records don't have month/year fields. Stamp them once so the
// per-month filters work retroactively.
```

```
    function migrateMonthStamps(){
        const tabs=['expenses','income','subscriptions','cconetime','ccinstall'];
        let stamped=0;
        for(const tab of tabs){
            const arr=_get(tab);
            let mutated=false;
            for(const r of arr){
                if(!r||r.month)continue;
                const ts=r.date_ms||r.date;
                const tb=bucket(ts);
                if(tb){r.month=tb.ym;r.year=tb.year;r.date_ms=tb.date_ms;mutated=
            }
            if(mutated)_set(tab,arr);
        }
        if(stamped>0)console.log('[allocator] migrated',stamped,'records with mon
        return stamped;
    }

    if(typeof document!=='undefined'){
        document.addEventListener('DOMContentLoaded',()=>{
            setTimeout(()=>{try{migrateMonthStamps();}catch(_){}}, 5500);
        });
    }

    console.log('[Allocator] ✓ WealthFlow Allocator v1.0 loaded');
})();
```